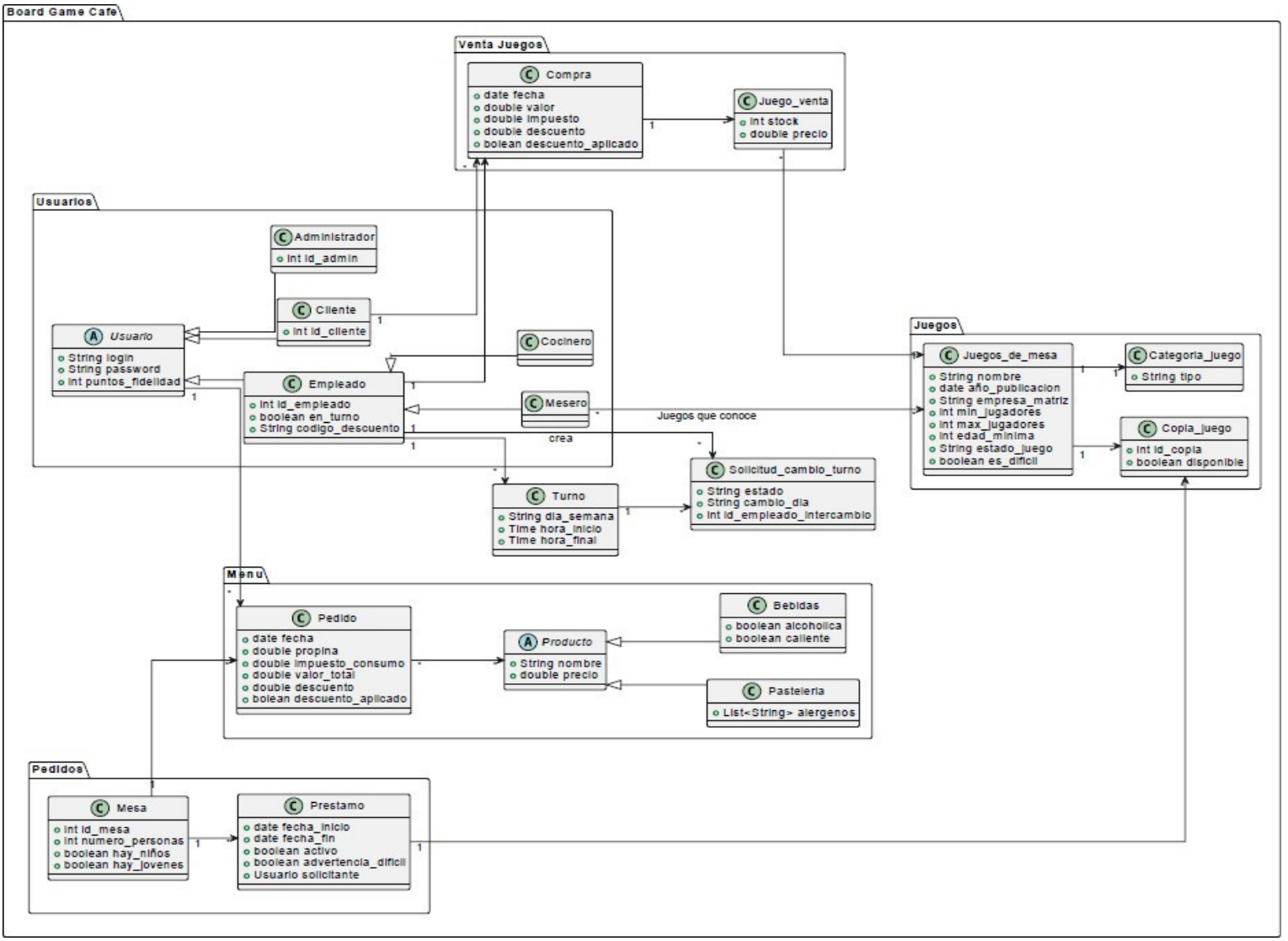
Entrega 2 del proyecto

Todo el diseño del sistema se realizó con el enfoque de hacer un manejo de datos claro y organizado que cuente con la capacidad de funcionar de manera eficiente para todo tipo de usuario en cualquier situación que se presente; por eso, nuestro sistema incluye múltiples funcionalidades como la gestión de usuarios, préstamos de juegos, ventas, turnos de empleados, restricciones operativas, registros y más.

Definición de entidades principales

Por un lado, una de las decisiones más importantes al momento de diseñar el sistema sería la definición de las entidades, ya que bajo estas mismas es que el mismo sistema va a interactuar entre sí, por ende, ningún aspecto puede quedar atrás, al igual que deben tener algún rol importante. Por ello, identificamos conceptos clave como lo serían Usuario, JuegoDeMesa, Mesa, Préstamo, Compra y Turno, los cuales cuentan con una alta cohesión, ya que cada clase tiene alguna responsabilidad dentro del sistema, lo cual terminaría creando la base de las clases.



Decisiones de diseño sobre el modelo

A partir del modelo, se tuvo que ir incorporando aspectos de funcionalidad y restricción que fueron generando cambios y ajustes sobre el UML. Las decisiones más relevantes fueron

BoardCafe

La implementación de clase controladora principal del sistema es el cambio más significativo en todo el uml, ya que es gracias a este que todo el sistema de la cafetería puede funcionar e interactuar con los distintos tipos de usuario. Esta clase termina funcionando como fachada al centralizar el acceso de todos los subsistemas evitando que cada módulo tenga que comunicarse directamente entre sí.

Asimismo, decidimos modelar la clase de BoardCafe como controlador del sistema ya que cuenta con todos los datos del sistema guardados en listas y mapas, al igual que cuenta con todas las operaciones que puede realizar cada tipo de usuario, con la finalidad de poder realizar las pruebas desde un único punto de entrada el cual facilita su acceso.

Juegos de mesa y Juegos de préstamo

Se decidió modelar los juegos de préstamo y de venta como entidades separadas. Por un lado un juego de préstamo es un objeto físico de la cafetería el cual cuenta con: copias individuales, historial de uso y disponibilidad en tiempo real. Mientras que un juego de venta solo necesita controlar el stock y precio, por ende si hubiesemos mezclado ambas categorías en una sola clase esto haría que contarán con atributos que no corresponden y haría más complejas las validaciones. Adicionalmente, esta separación permite sencillamente para el administrador pasar los juegos de un inventarios a otro con una operación concreta del sistema.

Copia del juego

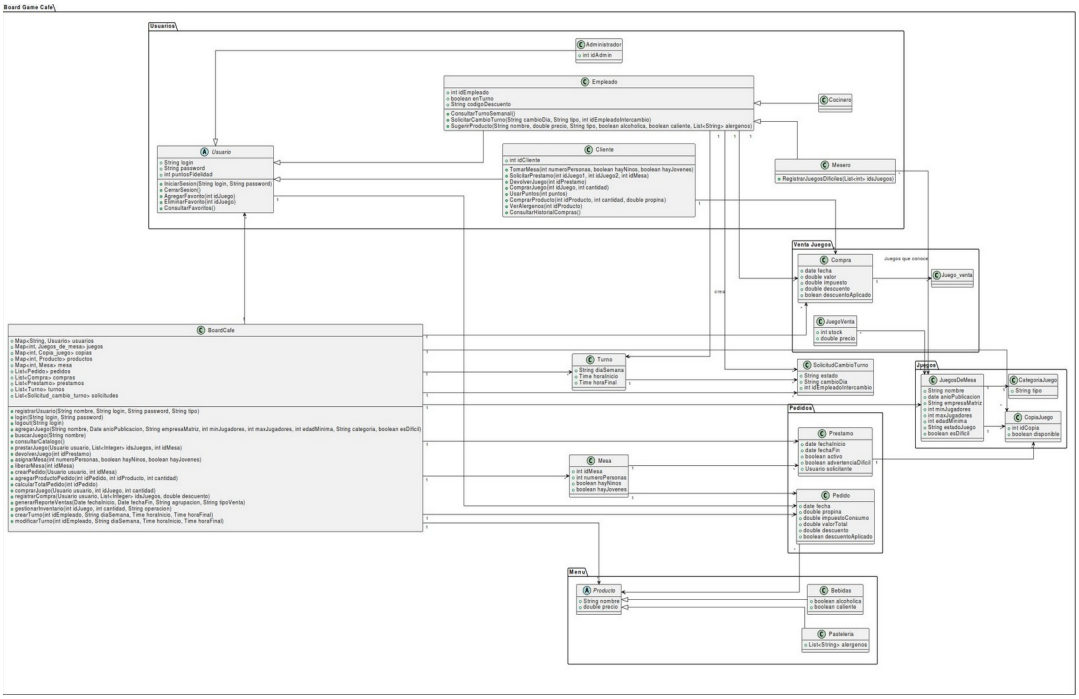
De igual forma, se decidió que cada copia física de un juego sea su propia entidad en lugar de manejar un contador en juegos_de_mesa. Esto permite revisar de forma individual el estado de cada copia, incluyendo su condición física, su historial de préstamos y las operaciones que se le pueden aplicar. Esto es relevante debido a que cumple con los requerimientos funcionales #21 y #24, los cuales mencionan que el administrador debe poder ver el historial completo de cada juego, al igual de poder marcar una copia específica como desaparecida sin tener que afectar a las demás, con lo cual nuestra propuesta cumple con dicha condición.

Préstamo

Se decidió que el préstamo sea una entidad propia y no un simple estado dentro de Mesa o Copia_juego. Esta decisión permite guardar la información completa de cada préstamo siendo: fecha de inicio, fecha de finalización, el usuario que lo solicitó y advertencia por si el juego es difícil. Al igual que para copia del juego, el requerimiento funcional #24 nos establece que el administrador debe poder consultar el historial completo de préstamos; por ende, si préstamo fuera únicamente un atributo booleano, toda esa información se perdería en el momento que el cliente devuelva el juego, lo que no permitirá cumplir dicho requerimiento.

Herencia de Usuario

Se decidió modelar los usuarios del sistema mediante herencias y subclases. Por un lado, Usuario es la clase abstracta que cuenta con sus tres subclases siendo Cliente, Empleado y Administrador, al igual que Empleado se divide entre Mesero y Cocinero. Esta decisión se tomó por dos razones: la primera es que todo usuario, sin importar su categoría, debe contar con su usuario y contraseña para acceder al sistema por ende todos heredan de la clase abstracta de Usuario. La segunda razón yace en que cada uno cuenta con un comportamiento y permisos distintos dentro del sistema, es por ello que se empiezan a dividir en los subgrupos para mantener una baja complejidad en las interacciones dentro del sistema. Adicionalmente, esta separación es vital debido a que se encuentran varios requisitos funcionales, los cuales son exclusivamente para cierto tipo de usuario y hacer validaciones en cada operación no sería eficiente.



(se adjunto la misma imagen pero con mejor calidad en la carpeta imágenes "uml completo")

Luego de realizar esta parte del **proyecto 1** hubo una retroalimentación por parte de los monitores que nos hizo caer en cuenta, los aspectos que se pueden mejorar de nuestro sistema para la cafetería. Es por ello, que ahora tocaba corregir errores e implementar sugerencias de la anterior entrega y agregar un nuevo sistema a la cafetería.

Torneos en la cafetería

Por un lado, la implementación más extensa fue agregar el sistema de torneos a toda la cafetería, en el cual se podían jugar tanto empleados fuera de turno como clientes en distintos tipos de torneo, lo cual hizo que se creará la clase de Torneo y clase de InscripcionTorneo con la finalidad de separar las tareas en diferentes secciones, lo cual permite mantener el sistema sencillo. Al igual, que para mantener sencillez se manejo el tipo de torneos mediante un string que indique que tipo de torneo es y de ahí se le aplican el resto de requerimientos.

Herencia de JuegoVenta

Nosotros decidimos que JuegoVenta herede de JuegosDeMesa, porque al momento de hacer una venta nos resulta muy útil tener acceso directo a atributos del juego como el nombre y la categoría, sin necesidad de hacer referencias adicionales. Es cierto que esta clase hereda cosas que no usamos directamente, como minJugadores o edadMinima, pero la simplicidad que nos aporta al registrar y mostrar una compra justifica esta decisión para el nivel de nuestro proyecto.

REC FUNC

Se implementaron más requerimientos funcionales debido a la información nueva que suministro de los torneos la cual se menciona más abajo, pero es un aspecto nuevo el cual se implementó y se debe tener en cuenta en la lógica de la cafetería.

Historias de Usuario

Se crearon la lista de las historias de usuarios sobre que se implementara en la aplicación en el casos de los 3 tipos de usuarios que tenemos en la cafetería, este se encuentra al final del documento el cual tiene sentido con el resto del desarrollo ya elaborado ya que con la estructura de la cafetería es que se maneja la funcionalidad con los usuario.

Interfaces

Luego, a partir del enunciado del Proyecto 2, nos piden hacer tres consolas main para los tres tipos de usuarios en la cafetería (cliente, empleado, administrador), en la cual cada uno pueda acceder y le permita hacer sus RF. Todo esto es relevante, ya que más adelante se le implementará la interfaz Swing, pero para esta entrega simplemente que aparezca en la consola está bien.

Pruebas JUnit

Asimismo, gran parte de la primera entrega se pedían simplemente las bases de cómo funciona el código, pero ahora se solicitó crear pruebas en JUnit, las cuales funcionan para cada clase individual con sus respectivas funciones. Lo cual hizo que se revisen de nuevo cada uno de los casos posibles al momento de acceder al sistema, los cuales deben cubrir cualquier tipo de error humano o de código.

Persistencias

La persistencia del sistema BoardCafe se implementó utilizando archivos JSON como medio de almacenamiento. La clase PersistenciaJSON centraliza toda la lógica de guardado y carga, operando sobre un objeto EstadoBoardCafe que actúa como contenedor de todas las colecciones y contadores del sistema. Cada vez que se realiza una operación que modifica el estado, como registrar un usuario, crear un torneo o realizar un pedido, BoardCafe invoca automáticamente el método guardarTodo(), el cual empaqueta el estado actual en el DTO y lo escribe en el archivo boardcafe_data.json

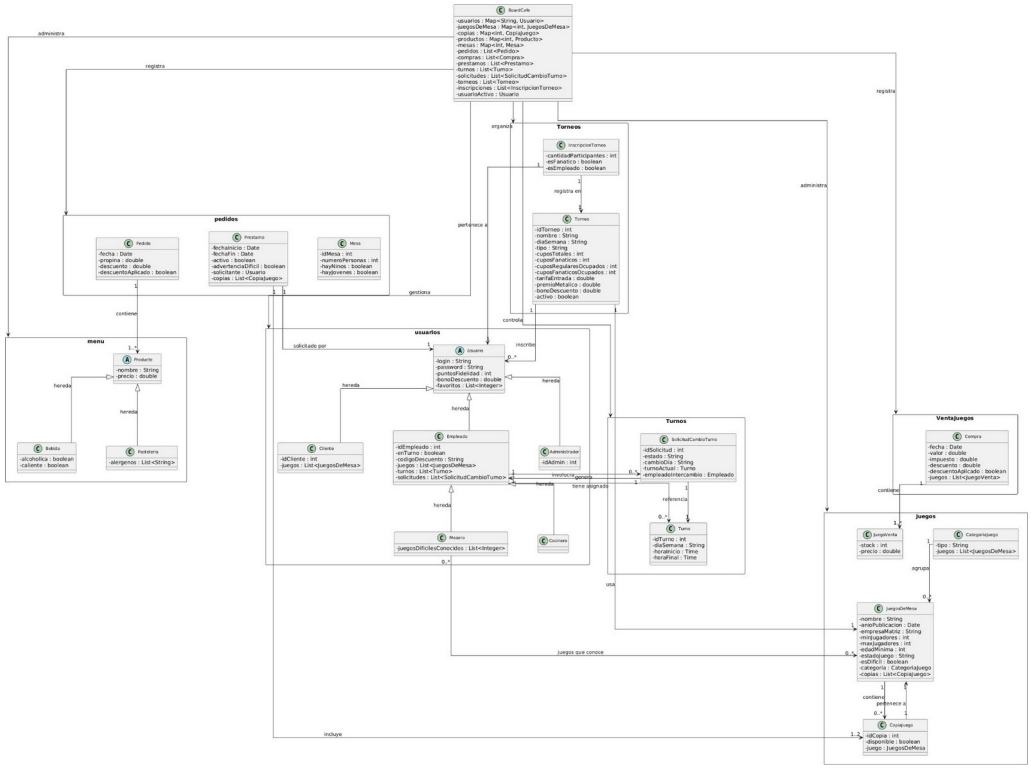
Para el desarrollo e implementación de la persistencia en este proyecto, se utilizó Claude como herramienta de apoyo, tomando como base el taller 3 de aerolíneas previamente trabajado en el curso. A partir de ese contexto, Claude analizó la estructura del código existente de BoardCafe.persistencias, identificó los patrones de diseño utilizados, corrijio las funciones implementadas en PersistenciaJSON y dio una solución coherente con la arquitectura orientada a objetos del proyecto.

UML

Por un lado, se tuvo que agregar un nuevo atributo llamado bonoDescuento en la clase de Usuario el cual permite guardar el premio obtenido de los torneos. De igual manera, para permitir más facilidad en identificar usuarios, compras, turno y solicitud de cambio se implementó el atributo de id el cual permite hacer la tarea de identificación mucha más fácil al momento de manejar datos. Asimismo ahora JuegoVenta hereda de JuegoDeMesa, y ya todas las relaciones cuentan con el nombre del proceso que llevan a cabo.

— Las imágenes se encuentran el carpeta imágenes las cuales permiten una mejor visualización–

Todo esto hizo que llegaramos al modelo actualizado del uml - el sencillo y el completo


BoardCafe
-usuarios : Map<String, Usuario>
-juegosDeMesa : Map<int, JuegosDeMesa>
-copias : Map<int, CopiaJuego>
-productos : Map<int, Producto>
-mesas : Map<int, Mesa>
-pedidos : List<Pedido>
-compras : List<Compra>
-prestamos : List<Prestamo>
-turnos : ListaTurno>
-solicitudes : List<SolicitudCambioTurno>
-torneos : List<Torneo>
-inscripciones : List<InscripcionTorneo>
-usuarioActivo : Usuario
administra
registra
organiza
Torneos
InscripcionTorneo
-cantidadParticipantes : int
-esFanatico : boolean
-esEmpleado : boolean
registra en
Torneo
-idTorneo : int
-nombre : String
-diaSemana : String
-horario : Time
-cuposTotales : int
-cuposFanaticos : int
-cuposRegularesOcupados : int
-cuposFanaticosOcupados : int
-tarifaEntrada : double
-premioMetalico : double
-bonoDescuento : double
-activo : boolean
pertenece a
gestiona
pedidos
Pedido
-fecha : Date
-propina : double
-descuento : double
-descuentoAplicado : boolean
Prestamo
-fechaInicio : Date
-fechaFin : Date
-activo : boolean
-advertenciaDifícil : boolean
-solicitante : Usuario
-copias : List<CopiaJuego>
Mesa
-idMesa : int
-numeroPersonas : int
-hayNiños : boolean
-hayJovenes : boolean
contiene
contabiliza
inscribe
usuarios
solicitado por
Usuario
-login : String
-password : String
-puntosFidelidad : int
-bonosDescuento : double
-favoritos : List<Integer>
hereda
Cliente
-idCliente : int
-juegos : List<JuegosDeMesa>
Empleado
-idEmpleado : int
-enTurno : boolean
-codigoDescuento : String
-juegos : List<JuegosDeMesa>
-turno : Turno
-solicitudes : List<SolicitudCambioTurno>
Administrador
-idAdmin : int
Mesero
-juegosDificilesConocidos : List<Integer>
Cocinero
interfaz
genera
Tiene asignado
Turnos
SolicitudCambioTurno
-idSolicitud : int
-estado : String
-cambioDia : String
-turnoActual : Turno
-empleadoIntercambio : Empleado
Turno
-idTurno : int
-diaSemana : String
-horaInicio : Time
-horaFinal : Time
referencia
usa
menu
Producto
-nombre : String
-precio : double
Bebida
-alcoholica : boolean
-caliente : boolean
Panaderia
-alergenos : List<String>
VentaJuegos
Compra
-fecha : Date
-valor : double
-impuesto : double
-descuento : double
-descuentoAplicado : boolean
-juegos : List<JuegoVenta>
contiene
Juegos
JuegoVenta
-stock : int
-precio : double
CategoriaJuego
-tipo : String
-juegos : List<JuegosDeMesa>
agrupa
JuegosDeMesa
-nombre : String
-anioPublicacion : Date
-empresaMatriz : String
-minJugadores : int
-maxJugadores : int
-edadMinima : int
-estadoJuego : String
-esDificil : boolean
-categoria : CategoriaJuego
-copias : List<CopiaJuego>
contiene
pertenece a
CopiaJuego
-idCopia : int
-disponible : boolean
-juego : JuegosDeMesa
Juegos que conoce
incluye

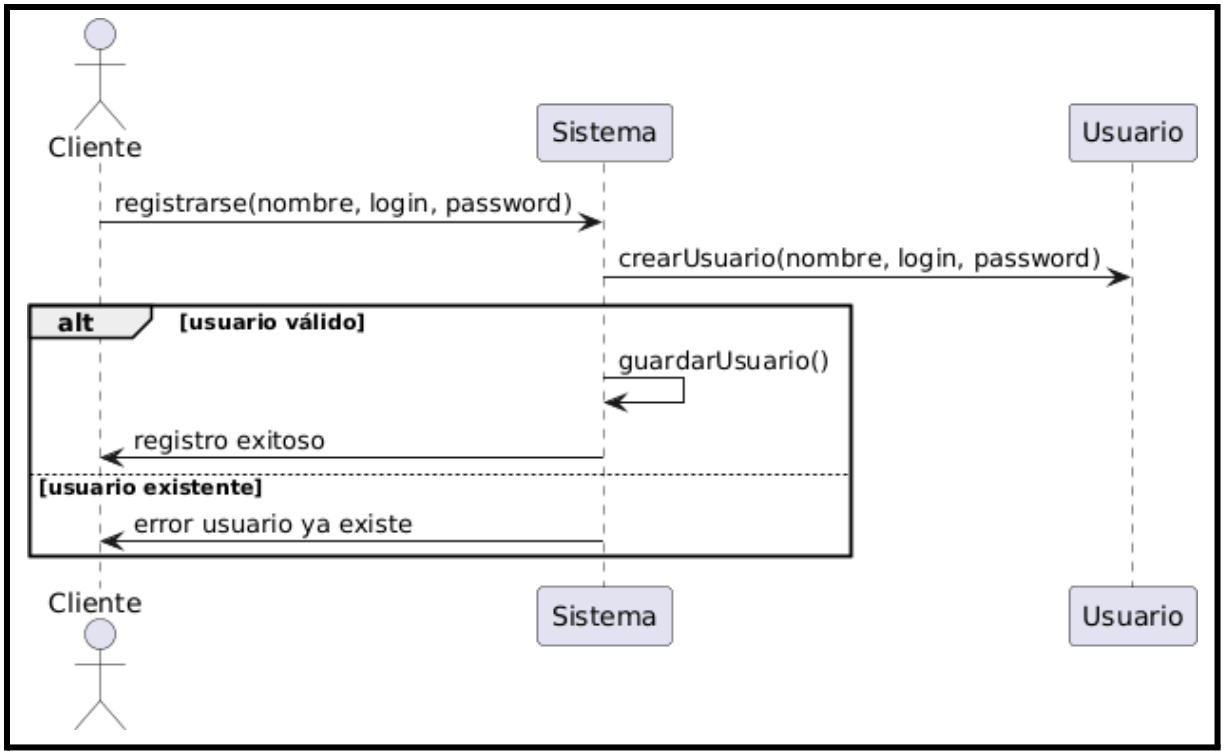
Sección funcional del sistema

1. ***Diagramas de flujo***
2. ***Requerimientos funcionales***
3. ***Explicación de las restricciones***

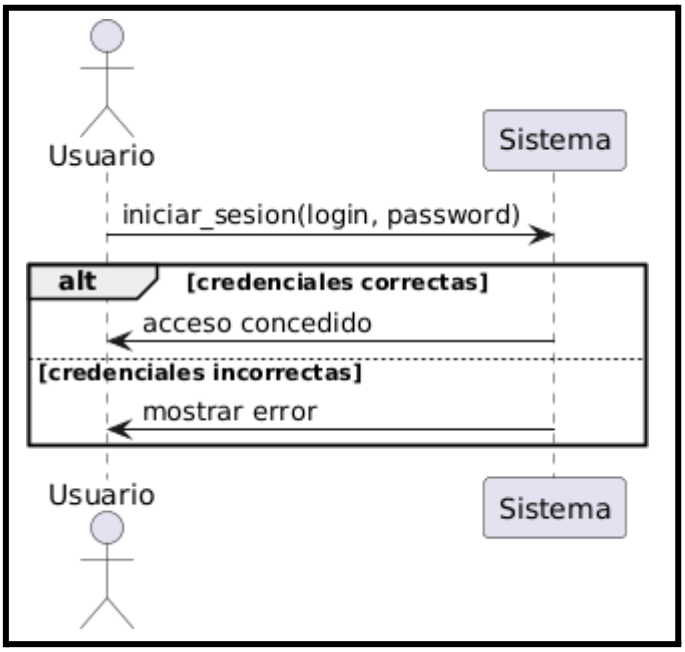
Luego, se tuvieron en consideración 8 requerimientos funcionales críticos los cuales permiten que el sistema exista operativamente (las que soportan el flujo principal del board-game café) los cuales serían los siguientes:

Acceso al sistema

1. **REQ 1 — Registrarse**

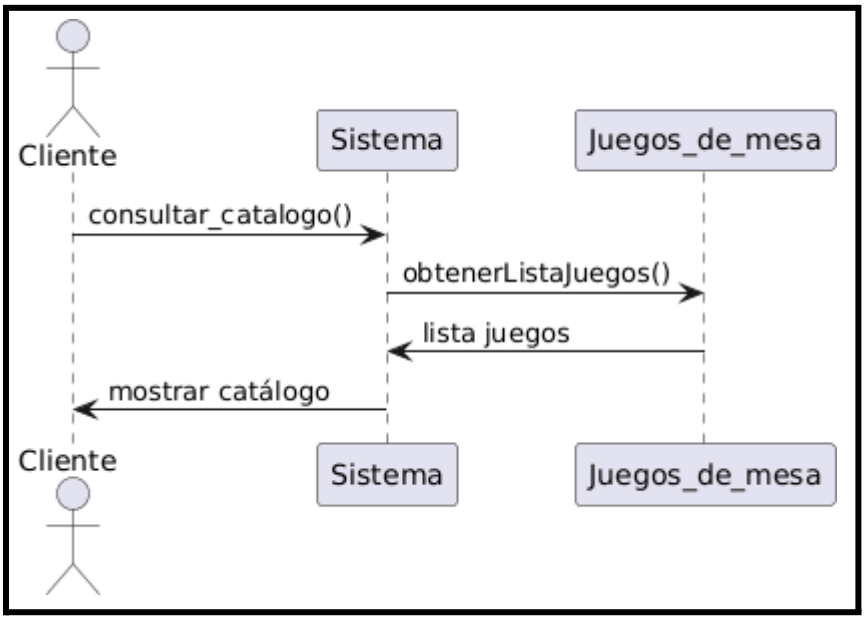


2. **REQ 2 — Iniciar sesión**

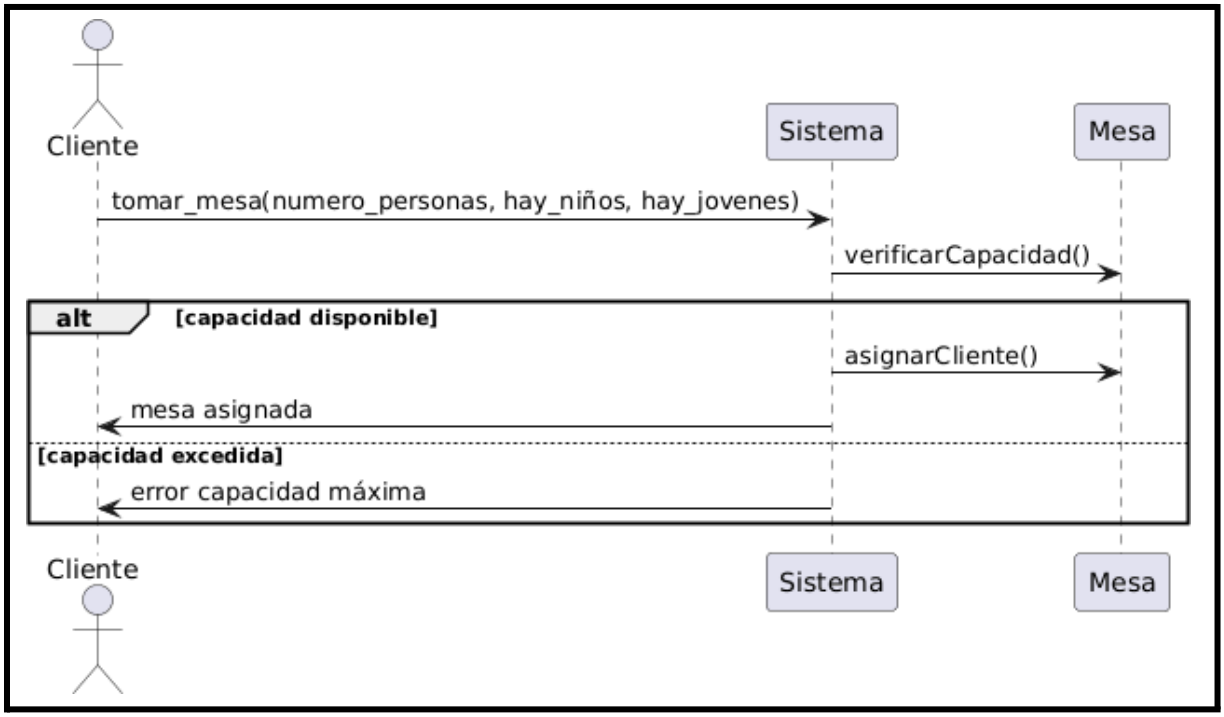


Operación principal del café

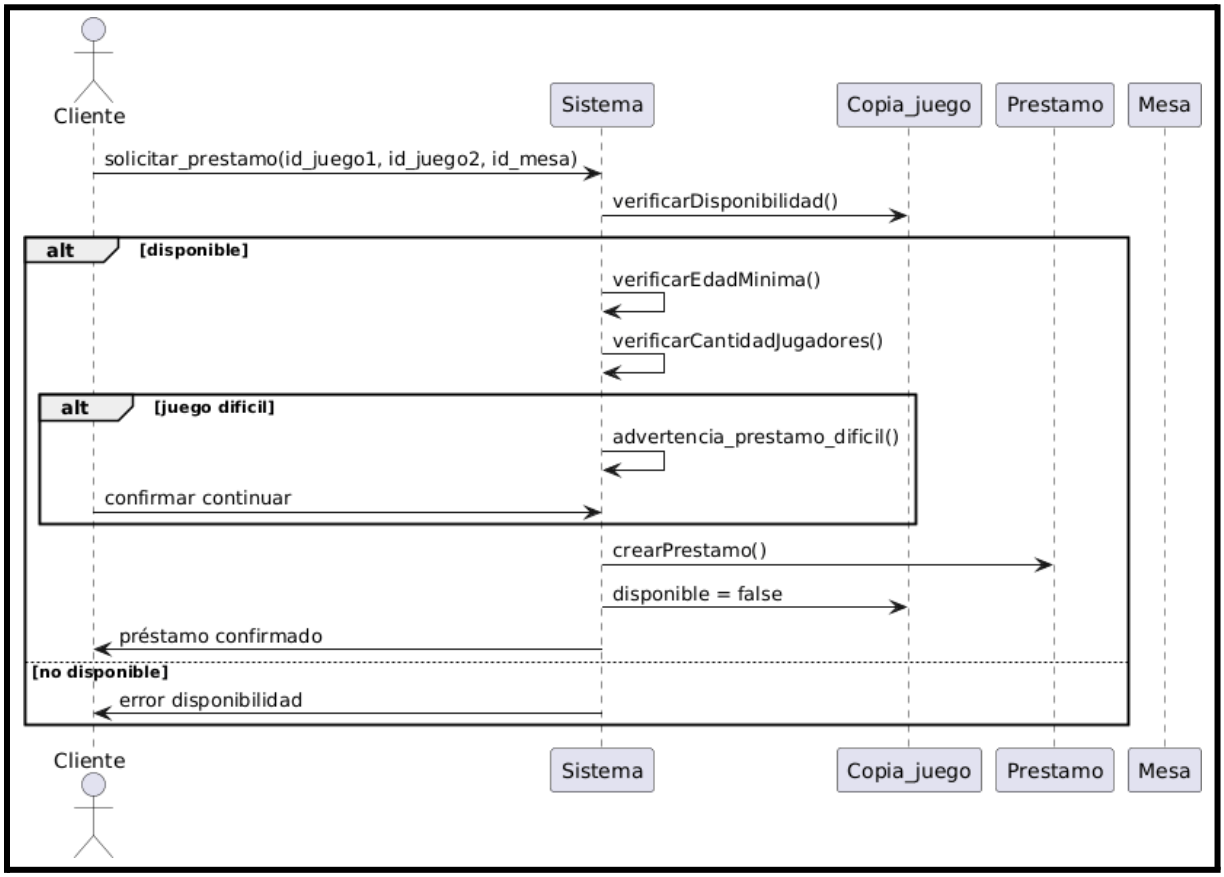
3. **REQ 3 — Consultar catálogo de juegos**



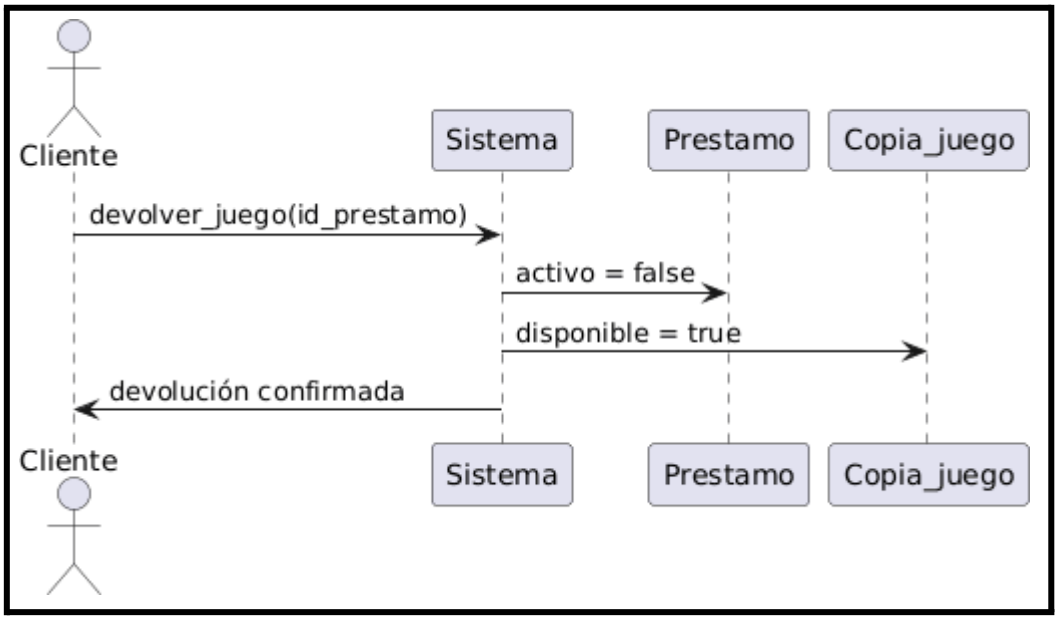
4. REQ 4 — Tomar mesa



5. REQ 5 — Solicitar préstamo de juegos

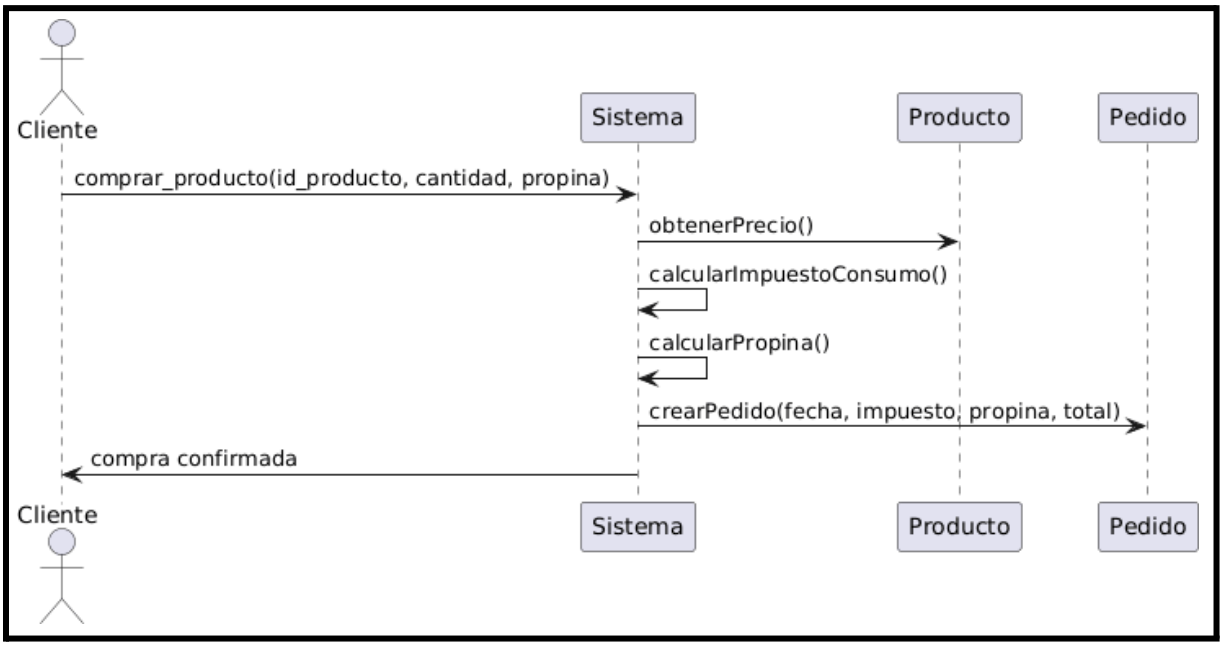


6. **REQ 7 — Devolver juegos**



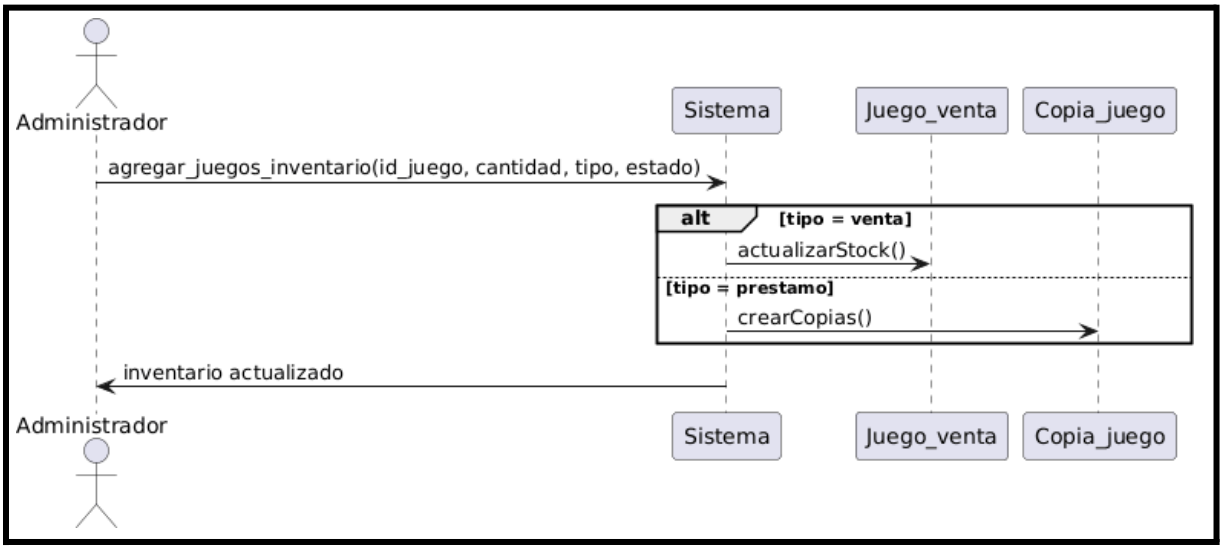
Operación comercial básica

7. **REQ 10 — Comprar productos cafetería**



Control mínimo del inventario

8. **REQ 22 — Agregar juegos al inventario**



Requerimientos funcionales que el sistema debe cumplir

REQ FUNCIONAL:	# 1
Usuario(s)	Cliente
Título	Registrarse en el sistema
Descripción	El sistema debe permitir al cliente crear una cuenta mediante su login y contraseña para acceder a todas las funcionalidades del board game cafe.
Datos de entrada:	String Nombre, String login, String password
Datos de salida:	Se crea la cuenta del cliente haciendo que se almacene, en otras palabras el Objeto usuario se crea y se almacena

REQ FUNCIONAL:	# 2
Usuario(s)	Cliente, Empleado, Administrador
Título	Iniciar sesión y cerrar sesión
Descripción	El sistema debe permitir a cualquier usuario poder acceder una vez coloque sus credenciales correspondientes, al igual que pueda cerrar su sesión al finalizar
Datos de entrada:	String login, String password
Datos de salida:	Hace que el usuario inicie sesión en su cuenta y si no existe o ingreso incorrectamente sus credenciales le aparecerá un mensaje de error indicando el problema

REQ FUNCIONAL:	# 3
Usuario(s)	Cliente, Empleado
Título	Consultar catálogo de juegos
Descripción	El sistema debe permitirle a los clientes y empleados poder ver el catálogo completo de los juegos que estén disponibles para préstamo, asimismo mostrándoles nombre, categoría, rango de jugadores, edad mínima, dificultad y si hay copias del juego disponible.
Datos de entrada:	Ninguno
Datos de salida:	La lista de los juegos de mesa con sus atributos y disponibilidad

REQ FUNCIONAL:	# 4
Usuario(s)	Cliente
Título	Tomar una mesa
Descripción	El sistema debe permitir al cliente reservar una mesa, en el cual debe indicar el número de personas, así como si hay niños o jóvenes, ya que con esta información el sistema verifica que no se supere la capacidad máxima del establecimiento antes de asignar la mesa.

Datos de entrada:	Int número de personas, boolean hay_niños, boolean hay_jovenes
Datos de salida:	El sistema le asigna la mesa al cliente y en caso que se supere el límite de capacidad mandaría error informando la razón. En otras palabras, al objeto mesa se le asigna un cliente.

REQ FUNCIONAL:	# 5
Usuario(s)	Cliente, Empleado
Título	Solicitar préstamo de juegos
Descripción	El sistema debe permitir a clientes y empleados solicitar hasta dos juegos de mesa simultáneamente. Para ello, el sistema validará la disponibilidad de los juegos y verificará que no se generen conflictos con base en las características de la reserva.
Datos de entrada:	ID(s) de Juegos_de_mesa solicitado, ID de la mesa
Datos de salida:	El sistema registra el préstamo y actualiza el estado del juego a activo = true, indicando que se encuentra en uso. Si la solicitud no puede completarse, el sistema genera un mensaje de error que informa dependiendo la razón (disponibilidad, edad, número de jugadores o presencia de bebidas calientes en juegos de acción)

REQ FUNCIONAL:	# 6
Usuario(s)	Cliente, Empleado
Título	Advertencia por juego difícil sin mesero capacitado
Descripción	El sistema debe informarle al cliente cuando el juego que solicita está dentro de la categoría “Difícil” (es_dificil = True) y no hay mesero capacitado para enseñar las reglas, el cual permite continuar con la solicitud pero con la advertencia del préstamo (advertencia_dificil = true).
Datos de entrada:	ID del juego dificil
Datos de salida:	Manda un mensaje advirtiendo que no hay mesero capacitado, por ende le pregunta al usuario que si quiere continuar y en el caso que acepte crearía la reserva con (advertencia_dificil = true).

REQ FUNCIONAL:	# 7
Usuario(s)	Cliente
Título	Devolver juegos prestados
Descripción	El sistema debe permitir al cliente registrar la devolución de los juegos al finalizar su estancia (activo=false), liberando las copias para que otros clientes puedan solicitarlas (disponible = true).
Datos de entrada:	ID del préstamo activo
Datos de salida:	Termina el préstamo (activo=false) y vuelve a poner disponible el juego (disponible = true).

REQ FUNCIONAL:	# 8
Usuario(s)	Cliente, Empleado
Título	Comprar juegos de mesa
Descripción	El sistema debe permitir al usuario comprar uno o más juegos de mesa disponibles en stock. No obstante, el cálculo de impuestos, descuentos y puntos de fidelidad dependerá del tipo de usuario que realice la compra.
Datos de entrada:	ID del usuario, ID(s) de los juegos y cantidad
Datos de salida:	Registro de la compra, total a pagar, confirmación del pago, puntos generados y stock actualizado.

REQ FUNCIONAL:	# 9
Usuario(s)	Cliente
Título	Usar puntos de fidelidad como descuento
Descripción	El sistema debe permitir a los clientes poder aplicar sus puntos de fidelidad al momento de hacer una compra.
Datos de entrada:	int puntos de fidelidad
Datos de salida:	Descuento aplicado en la compra o pedido realizado para luego actualizar los puntos de cliente a su nuevo valor

REQ FUNCIONAL:	# 10
Usuario(s)	Cliente, Empleado
Título	Comprar productos de la cafetería
Descripción	El sistema debe permitir ordenar una bebida o de la pastelería en el cual le aplique el impuesto de consumo y propina.
Datos de entrada:	ID del producto, int cantidad, double propina deseada
Datos de salida:	Retorna el valor del pedido con su fecha, propina, impuesto, descuento y valor total.

REQ FUNCIONAL:	# 11
Usuario(s)	Cliente, Empleado
Título	Alerta por alergenos
Descripción	El sistema debe informar los alergenos de un objeto de Pastelería antes de confirmar la compra.
Datos de entrada:	ID del objeto de pastelería
Datos de salida:	Lista de alérgenos asociados al producto de pastelería seleccionado.

REQ FUNCIONAL:	# 12
Usuario(s)	Cliente
Título	Consultar historial de compras
Descripción	El sistema debe permitir al cliente ver todas las compras que ha hecho anteriormente, mostrando la fecha y los productos.
Datos de entrada:	Ninguno
Datos de salida:	Lista de compras del cliente con fecha, productos, gasto total.

REQ FUNCIONAL:	# 13
Usuario(s)	Empleado
Título	Consultar turno semanal
Descripción	El sistema debe permitir al empleado ver su turno asignado con el día semana, hora_inicio y hora_final.
Datos de entrada:	Ninguno
Datos de salida:	Retorna el objeto turno del empleado con el día y el rango de las horas que debe trabajar

REQ FUNCIONAL:	# 14
Usuario(s)	Empleado
Título	Solicitar cambio de turno
Descripción	El sistema debe permitir al empleado poder crear una solicitud cambio turno mirando si es cambio o intercambio, al igual que verifique que no se incumpla el mínimo de 1 cocinero 2 meseros simultaneos
Datos de entrada:	String cambio_dia, String tipo, int id_empleado_intercambio
Datos de salida:	Retorna el objeto de la solicitud_cambio_turno con estado=pendiente o simplemente sería rechazado si incumple el mínimo de personal simultáneo.

REQ FUNCIONAL:	# 15
Usuario(s)	Empleado
Título	Sugerir un platillo al menú
Descripción	El sistema debe permitir a los empleados proponer un nuevo

	producto (Bebidas o Pastelería) para el menú, el cual quede pendiente hasta que el administrador lo apruebe
Datos de entrada:	String nombre, double precio, tipo Bebida/Pasteleria, atributos (alcohólica, alérgenos, caliente)
Datos de salida:	Sugiere el producto registrado como pendiente

REQ FUNCIONAL:	# 16
Usuario(s)	Mesero
Título	Registrar los juegos difíciles que conoce las reglas
Descripción	El sistema debe permitir al mesero registrar y actualizar la lista de juegos_de_mesa con es_dificil = true que conoce, ya que así puede asesorar a los clientes
Datos de entrada:	ID(s) de juegos_de_mesa que tengan es_dificil=true
Datos de salida:	Se actualiza la lista de juegos conocidos por el mesero

REQ FUNCIONAL:	# 17
Usuario(s)	Administrador
Título	Consultar informe de ventas
Descripción	El sistema debe permitir al administrador consultar un resumen de todas las ventas realizadas, con la posibilidad de visualizarlas por día, semana o mes. Además, el informe debe separar las ventas de juegos de mesa y las del café, mostrando en cada caso el subtotal, los impuestos cobrados y la propina correspondiente.
Datos de entrada:	Rango de fechas, agrupación deseada (día, semana, mes), tipo de venta (juegos, café, ambos)
Datos de salida:	Retorna el informe de ventas con el por subtotal, impuestos y propinas, separado por tipo de venta y agrupado según el periodo elegido.

REQ FUNCIONAL:	# 18
Usuario(s)	Administrador
Título	Aprobar o rechazar sugerencias de platillos
Descripción	El sistema debe permitir al administrador revisar las sugerencias de nuevos productos propuestos por los empleados y poderlas aprobar o rechazar.
Datos de entrada:	ID de la sugerencia seleccionada y decisión del administrador (aprobar o rechazar).
Datos de salida:	Actualiza la sugerencia, tanto la rechaza y elimina la sugerencia o la acepta y se actualiza el menú con el nuevo producto

REQ FUNCIONAL:	# 19
Usuario(s)	Administrador
Título	Aprobar o rechazar solicitudes de cambio de turno
Descripción	El sistema debe permitir al administrador revisar las solicitudes de cambio de turno con estado=pendiente y aprobarlos o rechazarlos.
Datos de entrada:	ID de Solicitud_cambio_turno, String decision (aprobar/rechazar)
Datos de salida:	Actualiza la solicitud de cambio, si la aprueba se modifica el turno y si la rechaza simplemente se mantiene igual.

REQ FUNCIONAL:	# 20
Usuario(s)	Administrador
Título	Crear y modificar los turnos semanales
Descripción	El sistema debe permitir al administrador crear y modificar objetos turno para cada empleado, verificando el mínimo de 1 Cocinero y 2 Meseros simultaneos.
Datos de entrada:	ID del Empleado, String dia_semana, Time hora_inicio, Time hora_final
Datos de salida:	Crea el objeto de turno o lo modifica y en el caso que incumpla el

	mínimo de persona manda error

REQ FUNCIONAL:	# 21
Usuario(s)	Administrador
Título	Marca copia desaparecida o robada
Descripción	El sistema debe permitir al administrador cambiar el estado de juego a desaparecido en el caso que no haya sido devuelto.
Datos de entrada:	Id de la copia_juego no devuelto
Datos de salida:	La copia del juego apareceria como estado_juego = Desaparecido, disponible = false y activo = false

REQ FUNCIONAL:	# 22
Usuario(s)	Administrador
Título	Agregar juegos al inventario
Descripción	El sistema debe permitir al administrador registrar la compra de nuevos juegos en juego_venta o copia_juego.
Datos de entrada:	ID de Juegos_de_mesa, int cantidad, Tipo (venta o préstamo), String estado_juego inicial
Datos de salida:	Actualiza el inventario de los juegos con las nuevas unidades adquiridas

REQ FUNCIONAL:	# 23
Usuario(s)	Administrador
Título	Mover juegos del inventario de venta al del préstamo y viceversa
Descripción	El sistema debe permitir al administrador trasladar un juego ubicado en Juego_venta y pasarlo a copia_juego y viceversa.
Datos de entrada:	ID del juego, String tipo (juego_venta o copia_juego), int cantidad

	de copias a mover
Datos de salida:	Actualiza el stock de cada juego según el movimiento que se realizó

REQ FUNCIONAL:	# 24
Usuario(s)	Administrador
Título	Ver inventario completo con historial de préstamos
Descripción	El sistema debe mostrar al administrador todos los objetos de copia_juegos con su estado, al igual que los préstamos que han tenido y el historial de cada prestamo(Usuario, fecha inicio, fecha fin).
Datos de entrada:	Ninguno
Datos de salida:	Retorna la lista de copia_juegos con sus atributos al igual que el contador de préstamo con el historial detallado

REQ FUNCIONAL:	# 25
Usuario(s)	Cliente, empleado
Título	Lista de juegos favoritos
Descripción	El sistema debe permitir a clientes y empleados agregar o eliminar juegos de mesa de su lista personal de favoritos.
Datos de entrada:	ID del Juegos_de_mesa a agregar o eliminar
Datos de salida:	Lista de favoritos del Usuario actualizada

Nuevos RF del proyecto 2

REQ FUNCIONAL:	**# 26**
Usuario(s)	Administrador
Título	Crear torneo

Descripción	El sistema debe permitir al administrador crear un torneo definiendo el juego, día, tipo (amistoso o competitivo), cupos y tarifa si aplica.
Datos de entrada:	ID del juego, String día, String tipo, int cupos, double tarifa
Datos de salida:	Torneo creado y registrado con sus cupos fanáticos calculados automáticamente

REQ FUNCIONAL:	**# 27**
Usuario(s)	Cliente, Empleado
Título	Inscribirse a un torneo
Descripción	El sistema debe permitir inscribirse en torneos. El cliente puede inscribir hasta 3 participantes. El empleado solo puede hacerlo si no tiene turno ese día y no recibirá premio en torneos competitivos.
Datos de entrada:	ID del torneo, int cantidad de participantes
Datos de salida:	Inscripción registrada con los cupos correspondientes actualizados

REQ FUNCIONAL:	**# 28**
Usuario(s)	Cliente, Empleado
Título	Desinscribirse de un torneo
Descripción	El sistema debe permitir al usuario eliminar su inscripción de un torneo, liberando todos los cupos tomados de una sola vez.
Datos de entrada:	ID del torneo
Datos de salida:	Inscripción eliminada y contadores de cupos actualizados

REQ FUNCIONAL:	**# 29**
Usuario(s)	Cliente, Empleado
Título	Consultar torneos disponibles
Descripción	El sistema debe permitir ver el listado de torneos activos mostrando nombre, tipo, juego, día, cupos disponibles y tarifa si aplica.
Datos de entrada:	Ninguno
Datos de salida:	Lista de torneos activos con sus atributos y disponibilidad de cupos

REQ FUNCIONAL:	**# 30**
Usuario(s)	Sistema
Título	Gestión de cupos fanáticos
Descripción	El sistema debe reservar automáticamente el 20% de los cupos para fanáticos del juego. Si estos se agotan, los fanáticos toman cupos regulares.
Datos de entrada:	Cupos totales del torneo, lista de favoritos del usuario
Datos de salida:	Cupos fanáticos y regulares actualizados según el usuario inscrito

REQ FUNCIONAL:	**# 31**
Usuario(s)	Administrador
Título	Registrar ganador del torneo
Descripción	El sistema debe permitir registrar al ganador de un torneo. Si es amistoso se asigna un bono de descuento al ganador. Si es competitivo se registra el premio en metálico y el torneo se marca como finalizado.
Datos de entrada:	ID del torneo, String login del ganador

Datos de salida:	Bono o premio registrado según el tipo de torneo, torneo marcado como activo = false

REQ FUNCIONAL:	**# 32**
Usuario(s)	Cliente
Título	Usar bono de descuento de torneo
Descripción	El sistema debe permitir al cliente aplicar el bono obtenido en un torneo amistoso en cualquier compra o pedido. No es acumulable con otros descuentos y se elimina una vez usado.
Datos de entrada:	ID de la compra o pedido donde se aplicará el bono
Datos de salida:	Descuento aplicado y bono del usuario actualizado a 0

Explicación de las restricciones

El sistema del **Board Game Cafe** debe cumplir las siguientes restricciones técnicas y funcionales:

Restricciones técnicas:

- La aplicación debe estar desarrollada en Java.
- Toda la información debe ser persistente.
- Los archivos deben guardarse en una carpeta diferente a la del código fuente.
- La aplicación se encarga de leer y escribir los archivos, por lo que ningún usuario modificara los archivos manualmente.
- Cada usuario del sistema debe tener un login y una contraseña para acceder
- En esta entrega no se requerirá interfaz gráfica.

Restricciones de los juegos:

- Cada juego pertenece **únicamente a una categoría:**
  - Cartas
  - Tablero

- Acción

- Si un juego tiene varias categorías se asignará según la prioridad.
    - Acción > Tablero > Cartas

- Los clientes sólo pueden pedir máximo dos juegos al mismo tiempo.
- Para pedir juegos, los clientes deben tener una mesa.
- Los empleados pueden pedir juegos sin mesa, solo si no están en turno o no hay clientes que atender.
- Los juegos no pueden prestarse si no hay copias disponibles.
- No se puede prestar un juego si el número de jugadores no es válido o la edad de los jugadores no cumple con la necesaria.

Restricciones de los juegos Difíciles:

- Los juegos difíciles requieren que un mesero explique las reglas.
- Cada mesero tiene una lista de juegos que sabe enseñar.
- Si ningún mesero conoce el juego, el préstamo se realiza pero el sistema muestra una advertencia.

Restricciones de la cafetería:

- El menú se divide en bebidas y pastelería.
- Las bebidas alcohólicas no pueden venderse en mesas con menores de edad
- Las bebidas calientes no pueden estar en mesas con juegos de acción
- Los productos de pastelería deben registrar alérgenos para advertir a los clientes.

Restricciones de los empleados:

- Existen dos tipos de empleados, cocineros y meseros.
- No pueden cambiar roles
- Para operar el café debe haber, mínimo, un cocinero y dos meseros.
- Los cambios de turno, deben ser solicitados por empleados y aprobados por el administrador.

Restricciones de ventas:

- Las ventas de juegos incluyen IVA del 19%.
- Las ventas del café incluyen, impuesto al consumo del 8%, y una propina opcional.
- Cada compra genera 1% en puntos de fidelidad.
- 1 punto = 1 peso de descuentos en compras futuras.
- Los empleados tienen un 20% de descuento.
- Los clientes pueden recibir 10% de descuento si usan el código de un empleado
- Los descuentos no son acumulables

Descripción de lo que demostrará los programas de prueba

Programa de prueba 1: Gestión de usuarios

Este programa demostrará:

- Registro de clientes (REQ 1)
- Inicio y cierre de sesión (REQ 2)
- Manejo de credenciales incorrectas
- Persistencia de usuarios
- Gestión de lista de favoritos (REQ 25)

Programa de prueba 2: Catálogo de juegos

Este programa demostrará:

- Consulta del catálogo de juegos (REQ 3)
- Visualización de atributos de los juegos
- Disponibilidad de juegos
- Gestión del inventario de juegos (REQ 22)

Programa de prueba 3: Mesas y préstamos

Este programa demostrará:

- Asignación de mesas (REQ 4)
- Solicitud de préstamo de juegos (REQ 5)

- Validaciones de:
    - número de jugadores
    - restricciones de edad
    - disponibilidad
    - máximo dos juegos
- Advertencias de juegos difíciles (REQ 6)
- Devolución de juegos (REQ 7)

Programa de prueba 4: Ventas y fidelización

Este programa demostrará:

- Compra de juegos (REQ 8)
- Compra de productos de cafetería (REQ 10)
- Cálculo de impuestos, propinas y descuentos
- Uso de puntos de fidelidad (REQ 9)
- Generación de puntos
- Consulta de historial de compras (REQ 12)

Programa de prueba 5: Restricciones de cafetería

Este programa demostrará:

- Restricción de bebidas alcohólicas con menores
- Restricción de bebidas calientes con juegos de acción
- Alerta de alérgenos (REQ 11)

Programa de prueba 6: Turnos de empleados

Este programa demostrará:

- Consulta de turnos (REQ 13)
- Solicitud de cambio o intercambio (REQ 14)
- Validación del mínimo de empleados
- Aprobación o rechazo de solicitudes (REQ 19)
- Creación y modificación de turnos (REQ 20)

Programa de prueba 7: Gestión del menú

Este programa demostrará:

- Sugerencia de nuevos platillos (REQ 15)
- Aprobación o rechazo de sugerencias (REQ 18)

Programa de prueba 8: Juegos difíciles

Este programa demostrará:

- Registro de juegos difíciles por meseros (REQ 16)
- Validación de disponibilidad de mesero capacitado
- Generación de advertencias en caso necesario

Programa de prueba 9: Administración e inventario

Este programa demostrará:

- Consulta de informe de ventas (REQ 17)
- Marcar juegos como desaparecidos (REQ 21)
- Movimiento de juegos entre inventarios (REQ 23)

- Visualización de inventario e historial de préstamos (REQ 24)

Historias de Usuario

Historia de usuario #1 **Registrarse en el sistema**

Como cliente quiero poder crear una cuenta con mi nombre, login y contraseña para acceder a todas las funcionalidades del Board Game Café.

Entradas: nombre, login, password

Salidas: Cuenta creada y almacenada en el sistema

Historia de usuario #2 **Iniciar y cerrar sesión**

Como cliente, empleado o administrador quiero poder ingresar al sistema con mis credenciales y cerrar sesión al finalizar.

Entradas: login, password

Salidas: Acceso concedido o mensaje de error si las credenciales son incorrectas

Historia de usuario #3 **Consultar catálogo de juegos**

Como cliente o empleado quiero ver el catálogo completo de juegos disponibles con su nombre, categoría, rango de jugadores, edad mínima, dificultad y disponibilidad.

Entradas: Ninguna

Salidas: Lista de juegos con sus atributos y disponibilidad de copias

Historia de usuario #4 **Tomar una mesa**

Como cliente quiero reservar una mesa indicando el número de personas y si hay niños o jóvenes para que el sistema me asigne una disponible.

Entradas: número de personas, boolean hayNinos, boolean hayJovenes

Salidas: Mesa asignada o mensaje de error si se supera la capacidad

Historia de usuario #5 **Solicitar préstamo de juegos**

Como cliente o empleado quiero solicitar hasta dos juegos simultáneamente para jugar en mi mesa, validando disponibilidad, edad y número de jugadores.

Entradas: ID(s) de juegos, ID de la mesa

Salidas: Préstamo registrado o mensaje de error según la restricción incumplida

Historia de usuario #6 **Advertencia por juego difícil**

Como cliente o empleado quiero ser avisado cuando el juego que solicito es difícil y no hay mesero capacitado para enseñarlo para decidir si continuar con el préstamo.

Entradas: ID del juego difícil

Salidas: Advertencia al usuario y préstamo registrado con advertenciaDificil = true si acepta

Historia de usuario #7 **Devolver juegos prestados**

Como cliente quiero registrar la devolución de los juegos al finalizar mi visita para liberar las copias y que otros clientes puedan usarlas.

Entradas: ID del préstamo activo

Salidas: Préstamo finalizado y copias marcadas como disponibles

Historia de usuario #8 **Comprar juegos de mesa**

Como cliente o empleado quiero comprar uno o más juegos disponibles en stock con el cálculo correspondiente de impuestos, descuentos y puntos de fidelidad.

Entradas: ID del usuario, ID(s) de juegos, cantidad

Salidas: Compra registrada, total calculado, puntos generados y stock actualizado

Historia de usuario #9 **Usar puntos de fidelidad**

Como cliente quiero aplicar mis puntos de fidelidad acumulados como descuento en una compra o pedido.

Entradas: puntos a usar, ID de la compra o pedido

Salidas: Descuento aplicado y puntos del usuario actualizados

Historia de usuario #10 **Comprar productos de la cafetería**

Como cliente o empleado quiero ordenar bebidas o pastelería con el cálculo del impuesto al consumo y propina opcional.

Entradas: ID del producto, cantidad, double propina

Salidas: Pedido registrado con fecha, impuesto, propina y total calculado

Historia de usuario #11 **Alerta por alérgenos**

Como cliente o empleado quiero ver los alérgenos de un producto de pastelería antes de confirmar mi pedido para tomar una decisión informada.

Entradas: ID del producto de pastelería

Salidas: Lista de alérgenos del producto seleccionado

Historia de usuario #12 **Consultar historial de compras**

Como cliente quiero ver todas mis compras anteriores con fecha, productos y gasto total.

Entradas: Ninguna

Salidas: Lista de compras con fecha, productos y valor total

Historia de usuario #13 **Consultar turno semanal**

Como empleado quiero ver mi turno asignado con el día, hora de inicio y hora de finalización.

Entradas: Ninguna

Salidas: Objeto turno con día y rango de horas

Historia de usuario #14 **Solicitar cambio de turno**

Como empleado quiero crear una solicitud de cambio o intercambio de turno, siempre que no se incumpla el mínimo de personal simultáneo.

Entradas: cambioDia, tipo, idEmpleadoIntercambio

Salidas: Solicitud creada con estado pendiente o rechazada si incumple el mínimo

Historia de usuario #15 **Sugerir platillo al menú**

Como empleado quiero proponer un nuevo producto al menú para que quede pendiente de aprobación por el administrador.

Entradas: nombre, precio, tipo, atributos según tipo

Salidas: Producto registrado con estado pendiente

Historia de usuario #16 **Registrar juegos difíciles que conoce**

Como mesero quiero registrar los juegos difíciles que sé enseñar para poder asesorar a los clientes que los soliciten.

Entradas: ID(s) de juegos con esDificil = true

Salidas: Lista de juegos conocidos por el mesero actualizada

Historia de usuario #17 **Consultar informe de ventas**

Como administrador quiero ver un resumen de ventas agrupado por día, semana o mes, separando juegos y productos de cafetería con subtotales, impuestos y propinas.

Entradas: Rango de fechas, agrupación deseada, tipo de venta

Salidas: Informe de ventas con subtotales, impuestos y propinas por tipo y periodo

Historia de usuario #18 **Aprobar o rechazar sugerencias de platillos**

Como administrador quiero revisar las sugerencias de nuevos productos propuestas por los empleados y decidir si las apruebo o rechazo.

Entradas: ID de la sugerencia, decisión (aprobar/rechazar)

Salidas: Producto agregado al menú si se aprueba, o eliminado si se rechaza

Historia de usuario #19 **Aprobar o rechazar cambios de turno**

Como administrador quiero revisar las solicitudes de cambio de turno pendientes y aprobarlas o rechazarlas.

Entradas: ID de la solicitud, decisión

Salidas: Turno modificado si se aprueba, o sin cambios si se rechaza

Historia de usuario #20 **Crear y modificar turnos semanales**

Como administrador quiero crear y modificar los turnos de los empleados verificando que siempre haya mínimo un cocinero y dos meseros simultáneos.

Entradas: ID del empleado, diaSemana, Time horaInicio, Time horaFinal

Salidas: Turno creado o modificado, o mensaje de error si incumple el mínimo

Historia de usuario #21 **Marcar copia desaparecida**

Como administrador quiero marcar una copia de juego como desaparecida cuando no ha sido devuelta, para actualizar el inventario.

Entradas: ID de la copia no devuelta

Salidas: Copia con estadoJuego , Desaparecido, disponible como false

Historia de usuario #22 **Agregar juegos al inventario**

Como administrador quiero registrar nuevos juegos en el inventario de venta o préstamo según corresponda.

Entradas: ID del juego, cantidad, tipo, estadoInicial

Salidas: Inventario actualizado con las nuevas unidades

Historia de usuario #23 **Mover juegos entre inventarios**

Como administrador quiero trasladar juegos del inventario de venta al de préstamo y viceversa según las necesidades del café.

Entradas: ID del juego, tipo origen, cantidad

Salidas: Stock de cada inventario actualizado según el movimiento

Historia de usuario #24 **Ver inventario completo con historial**

Como administrador quiero ver todas las copias de juegos con su estado actual y el historial completo de préstamos de cada una.

Entradas: Ninguna

Salidas: Lista de copias con atributos e historial de préstamos detallado

Historia de usuario #25 **Lista de juegos favoritos**

Como cliente o empleado quiero agregar o eliminar juegos de mi lista personal de favoritos para tenerlos identificados fácilmente.

Entradas: ID del juego a agregar o eliminar

Salidas: Lista de favoritos actualizada

Historia de usuario #26 **Crear torneo**

Como administrador quiero crear torneos definiendo el juego, día, tipo, cupos y tarifa para ofrecer nuevas actividades a los clientes.

Entradas: ID del juego, día, tipo, cupos, tarifa

Salidas: Torneo creado con cupos fanáticos calculados automáticamente

Historia de usuario #27 **Inscribirse a un torneo**

Como cliente o empleado quiero inscribirme en un torneo disponible, respetando el límite de participantes y las restricciones según mi tipo de usuario.

Entradas: ID del torneo, cantidad de participantes

Salidas: Inscripción registrada con cupos actualizados

Historia de usuario #28 **Desuscribirse de un torneo**

Como cliente o empleado quiero cancelar mi inscripción en un torneo liberando todos mis cupos de una sola vez.

Entradas: ID del torneo

Salidas: Inscripción eliminada y cupos del torneo actualizados

Historia de usuario #29 **Consultar torneos disponibles**

Como cliente o empleado quiero ver el listado de torneos activos con sus características para decidir en cuál inscribirme.

Entradas: Ninguna

Salidas: Lista de torneos activos con atributos y cupos disponibles

Historia de usuario #30 **Gestión de cupos fanáticos**

Como cliente o empleado fanático de un juego quiero tener prioridad en los cupos del torneo de ese juego antes que los demás usuarios.

Entradas: Lista de favoritos del usuario, cupos totales del torneo

Salidas: Cupo fanático o regular asignado según disponibilidad

Historia de usuario #31 **Registrar ganador del torneo**

Como administrador, quiero registrar al ganador de un torneo para asignarle el premio correspondiente y marcarlo como finalizado.

Entradas: ID del torneo, login del ganador

Salidas: Bono o premio registrado y torneo marcado como activo sea false

Historia de usuario #32 **Usar bono de descuento de torneo**

Como cliente, quiero aplicar el bono ganado en un torneo amistoso en cualquier compra o pedido sin acumularlo con otros descuentos.

Entradas: ID de la compra o pedido

Salidas: Descuento aplicado y bono del usuario actualizado a 0